

SISTEMA FORECAST DCIC SpA — Documentación Técnica

Segunda Sesión del Panel — 24 Junio 2026 | Arquitectura, APIs y Cambios de Implementación

1. STACK TECNOLÓGICO

Capa	Tecnología	Version / Nota
Backend	FastAPI + Uvicorn	0.111 / ASGI async
ORM	SQLAlchemy AsyncSession	2.0 — queries async con text() + bindparams
Driver BD	asyncpg	0.29 — JSONB: CAST(:x AS jsonb), no ::jsonb
BD	PostgreSQL	15 — schema forecast_dcic
Modelo	statsmodels ExponentialSmoothing	Holt-Winters add/add/12 — optimized=True
Migraciones	Alembic	env.py con create_engine() directo (sin config.set_main_option)
Frontend	React 18 + Vite	Puerto 3002 — CORS allow_origins exacto
Auth	JWT + API Key M2M	roles: admin, editor, viewer; M2M para ERP
HTTP Client	httpx (async)	timeout=10s en llamadas BCC — circuit-breaker manual

2. CONSTANTS.PY — FUENTE ÚNICA DE CONFIGURACIÓN

Nuevo archivo backend/constants.py centraliza todas las constantes configurables vía variables de entorno. Elimina magic numbers dispersos en 5+ archivos. Cada módulo importa desde constants.

Constante	Default	Env var	Usado en
IVA_FACTOR_FLOAT	1.19	—	forecast_2027.py, compras.py
USD_NEUTRO	870.0	USD_NEUTRO	crear_forecast_2027.py, tipo_cambio.py
ALERTA_UMBRAL_CLP	500000	ALERTA_UMBRAL_CLP	compras.py /alertas-rojo
PHI_CAP	0.03	PHI_CAP	crear_forecast_2027.py — CAGR_ADJ_MAX
MACRO_SENS	0.003	MACRO_SENS	crear_forecast_2027.py — factor_macro
HW_ALPHA	None (auto)	HW_ALPHA	modelo_holt_winters.py — .fit()
HW_BETA	None (auto)	HW_BETA	modelo_holt_winters.py — .fit()
HW_GAMMA	None (auto)	HW_GAMMA	modelo_holt_winters.py — .fit()

3. NUEVOS ENDPOINTS — V3

GET `/api/dashboard` auth: Libre | dashboard.py
KPIs ejecutivos consolidados: disponibilidad semaforo, MAPE/bias, compras pendientes CLP/USD, alertas criticas, tipo de cambio, ultimo sync ERP. Una sola llamada para el panel directivo.

GET `/api/compras/alertas-rojo` auth: Libre | compras.py
SKUs en semaforo ROJO cuyo importe de compra $\geq$ umbral (default ALERTA_UMBRAL_CLP=500.000 CLP). Param opcional: ?umbral=N

GET `/api/forecast-2027/hw-params` auth: Libre | forecast_2027.py
Retorna parametros HW actuales leidos desde env: alpha, beta, gamma, phi_cap, macro_sens.

PUT `/api/forecast-2027/hw-params` auth: admin | forecast_2027.py
Actualiza parametros HW en el archivo .env del backend. Requiere reinicio del servidor. Campos opcionales: alpha, beta, gamma (0-1 o null=auto), phi_cap (0-0.20), macro_sens (0-0.05).

POST `/api/tipo-cambio/sync-auto` auth: admin, editor | tipo_cambio.py
Circuit-breaker BCC (timeout=10s). Si BCC falla: verifica dato de los ultimos 3 días. Si no hay: inserta USD_NEUTRO con fuente=fallback_env. Degrada graciosamente sin error 5xx.

4. MODELO HOLT-WINTERS — PARAMETRIZACION COMPLETA

El modelo ExponentialSmoothing (statsmodels) es ahora completamente configurable sin tocar código. La cadena de configuración es: variable de entorno -> constants.py -> modelo_holt_winters.py -> .fit()

```
# constants.py
HW_ALPHA = float(os.getenv("HW_ALPHA", "0")) or None # 0 = auto
HW_BETA = float(os.getenv("HW_BETA", "0")) or None
HW_GAMMA = float(os.getenv("HW_GAMMA", "0")) or None

# modelo_holt_winters.py
HW_TREND = os.getenv("HW_TREND", "add") # add | mul
HW_SEASONAL = os.getenv("HW_SEASONAL", "add")
HW_PERIODS = int(os.getenv("HW_PERIODS", "12"))

modelo = ExponentialSmoothing(
    serie_clean, trend=HW_TREND, seasonal=HW_SEASONAL,
    seasonal_periods=HW_PERIODS, initialization_method="estimated",
).fit(
    optimized=True,
    smoothing_level=HW_ALPHA, # None -> statsmodels optimiza
    smoothing_trend=HW_BETA,
    smoothing_seasonal=HW_GAMMA,
)
```

Para sobrescribir los parámetros sin reiniciar, usar PUT /api/forecast-2027/hw-params (admin). El endpoint escribe en el .env del backend. El siguiente inicio del servidor aplica los cambios.

5. DASHBOARD EJECUTIVO — ESTRUCTURA DE RESPUESTA

GET /api/dashboard retorna un objeto JSON con 6 bloques. Sin parámetros requeridos. Calcula todo en tiempo real desde las tablas de producción.

```
{
  "generado_en": "2026-06-24",
  "disponibilidad": {
    "verde": { "cantidad": 412, "pct": 68.2 },
    "amarillo": { "cantidad": 98, "pct": 16.2 },
    "rojo": { "cantidad": 94, "pct": 15.6 },
    "total_skus_canal": 604
  },
  "modelo": {
    "mape_promedio": 18.4, "bias_promedio": -2.1,
    "skus_con_metrica": 287,
    "calidad": { "bueno": 201, "regular": 68, "alto": 18 }
  },
  "compras": {
```

```

"skus_rojo": 94,
"valor_compras_pendientes_clp": 48750000,
"valor_compras_pendientes_usd": 56035
},
"alertas": { "umbral_clp": 500000, "total_alertas": 12, ... },
"tipo_cambio": { "usd_clp": 870, "fuente": "bcc", "estado": "ok" },
"ultimo_sync": { "insertados": 1240, "estado": "completado" }
}

```

6. CIRCUIT-BREAKER TIPO DE CAMBIO

La logica de sync-auto implementa un patron circuit-breaker simple con tres niveles de degradacion. El sistema nunca lanza una excepcion 5xx al cliente por fallo de BCC.

Nivel	Condicion	Accion	fuelle en BD
1 — Ideal	BCC responde HTTP 200 con datos	Inserta/actualiza con ON CONFLICT	"bcc"
2 — Fallback soft	BCC falla pero hay dato de ultimos 3 dias	Retorna dato existente sin insertar	"existente"
3 — Fallback duro	BCC falla y no hay dato reciente	Inserta USD_NEUTRO del env	"fallback_env"

7. REFACTOR FRONTEND — TABLAFORES CAST.JSX

Archivo	Lineas	Responsabilidad
TablaForecast.jsx (antes)	1158	Todo mezclado: state, fetch, logica de negocio, render, filtros, sort, export
TablaForecast.jsx (V3)	834	Solo JSX de render + desestructuracion del hook
hooks/useForecastTabla.js	424	State, useEffect, handlers, useMemo (filas filtradas, ordenadas, opciones)
utils/forecastUtils.js	46	Constantes (MESES, COLS_FIJAS, LEFT_OFFSETS) y funciones puras (clp, mclp)

8. INVENTARIO DE ARCHIVOS MODIFICADOS — V3

Estado	Archivo	Cambio
NUEVO	backend/constants.py	Constantes globales centralizadas + env vars
NUEVO	backend/routers/dashboard.py	Endpoint GET /api/dashboard KPIs ejecutivos
NUEVO	frontend/src/.../hooks/useForecastTabla.js	Hook React con state y logica de negocio
NUEVO	frontend/src/.../utils/forecastUtils.js	Utilidades y constantes puras del forecast
MODIFICADO	backend/modelo_holt_winters.py	HW_TREND/SEASONAL/PERIODS desde env; alpha/beta/gamma desde constan
MODIFICADO	backend/crear_forecast_2027.py	PHI_CAP, MACRO_SENS importados de constants
MODIFICADO	backend/routers/ventas.py	Paginacion skip/limit; CAST JSONB; auto-retrain post-sync
MODIFICADO	backend/routers/compras.py	IVA desde constants; endpoint /alertas-rojo
MODIFICADO	backend/routers/forecast_2027.py	Endpoints GET/PUT /hw-params; import PHI_CAP, MACRO_SENS
MODIFICADO	backend/routers/tipo_cambio.py	Endpoint /sync-auto con circuit-breaker y fallback
MODIFICADO	backend/main.py	Registro de dashboard router
MODIFICADO	frontend/src/.../TablaForecast.jsx	Refactorizado para usar useForecastTabla hook